Parallel Random Forest - Model overview and experimental results

1 Model

Random forest is the model chosen for implementation. However, since this model relies on Decision Trees, they were implemented as well. Implementation for both was done in Python.

1.1 Decision Tree

The fitting process is done in the following way:

- recursively split each node based on the best feature, starting from the root node;
- at each node, entropy is calculate for each feature
- best feature is the one with the lowest entropy value;
- stopping criteria for the recursive algorithm include: entropy in a node is 0, max tree depth is reached, minimum number of samples is reached.

Before making a split, the program checks for the minimum number of samples to be present in the child nodes.

Feature importance is also computed. The more a feature reduces entropy, the more important it is.

Afterwards, for prediction, the tree is traversed for each entry in the test set and a prediction is returned for each. In the case where some feature values were not encountered during training, the prediction is the most common class in the current node's data.

The available hyperparameters are: max features to consider for split, max tree depth, minimum samples for split, minimum samples at leaf. Tree pruning is done using the max tree depth, minimum samples for split and minimum samples at leaf hyperparameters.

1.2 Random Forest

Random Forest is an ensemble of decision tree learners.

The model instantiates a number of decision trees, which will use bootstrap samples for fitting. A bootstrap sample is generated by selecting multiple data points with replacement from the data set. A process-based parallel implementation is used to speed up the process by fitting multiple trees concurrently and avoid the global interpreter lock. For a smaller number of decision trees, the fitting process is handled sequentially. However, when the number of trees is large, the parallel option is used, where the trees are divided into batches, with each process fitting a batch of, for example, 10 trees at a time. During the splitting process, each decision tree can consider a subset of features of size $m = \sqrt{d}$, where d is the number of features in the data set.

The final prediction of the model is an aggregation of the predictions of each decision tree. Since the learning task is a binary classification, the final prediction is the majority vote.

The available hyperparameters are: all the hyperparameters for decision trees, number of trees.

For selecting the best hyperparameter configuration, a grid search method is used. Combinations of hyperparameters are tested on the model, and the one which offers the best test accuracy is chosen.

2 Experimental results

Using the grid search algorithms, the following hyperparameter configuration was chosen:

- number of trees = 5
- max tree depth = 5
- minimum samples at leaf = 1
- minimum samples for a split = 2

These hyperparameters were used for both the library implementation and the one from scratch.

Data was split in the following way: 80% for training and 20% for test.

The used library implementation of Random Forest comes from the open source library scikit-learn, designed for machine learning and data mining. It offers an extensive set of features, including the option of multiple spliting criteria, class weighting, decision tree pruning, parallel tree fitting and so on. For comparison purposes, only the features present in the custom model as well will be utilized.

Metric	**Custom Random Forest**	**Library Random Forest**
Train Accuracy (%)	94.88	89.44
Test Accuracy (%)	87.23	89.97
Precision	0.90	0.90
Recall	0.97	1.00
F1-measure	0.93	0.95
5-fold Cross Validation Score	0.90	0.89
Standard Deviation	0.01	0.01
Confidence Interval (95%)	[0.90, 0.91]	[0.89, 0.90]
AUC	0.4844	0.4619
AUPRC	0.1055	0.0960

Table 1: Comparison between Custom Random Forest and Library Random Forest

We will begin the comparison between the two random forest models on the same dataset.

The test accuracy is higher for the library model than the custom model. Train accuracy is lower for the library implementation. The custom model fits the training data well, but the discrepancy between the train and test accuracy might suggest overfitting to some degree.

Precision of both models is 0.9, which indicates both models classify positive instances correctly.

The custom model has lower recall than the library models, which indicates the library model is better at identifying all relevant instances of the positive class.

Both models have a high F1-score, suggesting a good balance between precision and recall. The library model's f1-score is slightly higher.

5-fold Cross validation Score is the average score across the 5 folds. The custom implementation has a higher cross validation score, which suggests the custom model performs more consistently across different subsets of the data compared to the library model.

Both models' standard deviation is 0.01. This shows the performance is stable across the 5 folds.

The custom model has a slightly higher confidence interval range, which shows that its performance is more stable and confident.

Both models achieve a low AUC score, with the custom model achieving a slightly higher score. Their lack of ability to distinguish between classes might be the result of imbalanced target label.

The AUPRC score is close to 0 in both cases. This performance could be a consequence of data imbalance.

For model explainability, the LIME algorithm is employed. LIME explains model predictions by creating perturbed versions of the input instance and assigning weights based on proximity using a kernel function. A weighted linear regression is then fit, using the perturbed data as input, the original model's predictions as targets, and the proximity-based weights. The regression coefficients indicate the importance of each feature for the prediction around the instance.

Feature range	**Contribution Score**
Parental_Education_Level $>$ 1.00	0.000159
Previous_Scores $>$ 88.00	0.000137
65.00 $<$ Exam_Score $\leq$ 67.00	0.000133
3.00 $<$ Physical_Activity $\leq$ 4.00	0.000119
6.00 $<$ Sleep_Hours $\leq$ 7.00	0.000115
16.00 $<$ Hours_Studied $\leq$ 20.00	0.000114
Attendance $\leq$ 70.00	0.000111
1.00 $<$ Family_Income $\leq$ 2.00	8.070915e-05
1.00 $<$ Peer_Influence $\leq$ 2.00	7.659987e-05
Motivation_Level $\leq$ 1.00	6.434870e-05
Tutoring_Sessions $\leq$ 1.00	6.283388e-05
0.00 $<$ Access_to_Resources $\leq$ 2.00	5.479675e-05
1.00 $<$ Distance_from_Home $\leq$ 2.00	5.240081e-05
0.00 $<$ Gender $\leq$ 1.00	5.149043e-05
0.00 $<$ Extracurricular_Activities $\leq$ 1.00	5.058381e-05
0.00 $<$ School_Type $\leq$ 1.00	4.758478e-05
0.00 $<$ Teacher_Quality $\leq$ 2.00	4.481271e-05
0.00 $<$ Parental_Involvement $\leq$ 2.00	4.156668e-05
Internet_Access $\leq$ 1.00	0.0

Table 2: LIME Results

The most impactful features are parental education level, previous scores, exam scores, physical activity, sleep hours, hours studied and attendance. All features have a positive score, which indicates that when the value of a feature is within some range, for example when parental education level value is above 1, they will push the prediction towards the positive class. When their values are outside their ranges, the prediction will move towards the negative class.

Academic and lifestyle factors seem to have the greatest influence on the prediction of the learning disabilities label.